

CS383 - Software Engineering

Software Testing (II)

Outline

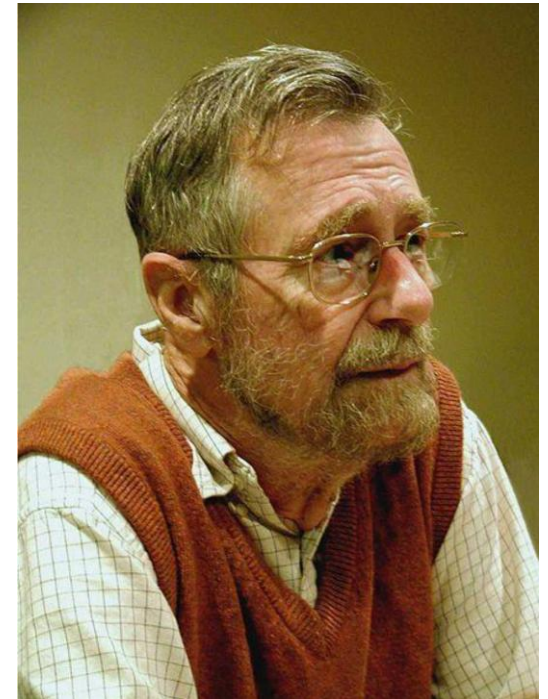
- Unit testing
- Unit testing approaches
- Unit testing techniques
 - Specification-based testing
 - Model-based testing
 - Boundary testing
 - Coverage-based testing

Unit testing

- The famous quote on testing...

“Program testing can be used to show the presence of bugs, but never to show their absence!” by Edsger W. Dijkstra

<http://en.wikiquote.org/wiki/Edsger_W._Dijkstra>



Unit testing approaches

- There are three approaches for unit testing
 - Black-box testing: developers do not necessary know about implementation details to do the testing.
 - Eg, testing a reused component
 - White-box testing: developers have access to the code and able to track the execution of a test.
 - Eg. testing to compute code coverage
 - Grey-box testing: developers have partial access to the code and able to track part of the execution of a test

Unit testing techniques

- There are many testing techniques exist, several of those are:
 - Specification-based testing
 - Model-based testing
 - Random testing (includes search-based testing)
 - Boundary testing
 - Coverage-based testing (important)
 - Fault injection (powerful testing approach)

Specification-based testing

- Testing software against all the claims made about it in the specification
 - Benefits
 - Systematic
 - Simpler than software implementation itself
 - Captures the aspects that matter the most about software
 - Drawbacks
 - Not all specification (like informal ones) can be converted to tests

Specification-based testing example

- Below is a specification test written in Spark (extension of Ada language)

```
procedure Increment
```

```
  (X : in out Integer)
```

```
with
```

```
  Global  => null, -- doesn't read global variables
```

```
  Depends => (X => X), -- value of X depends on itself
```

```
  Pre      => X < Integer'Last, -- less than the last
```

```
  -- possible value (to avoid overflow)
```

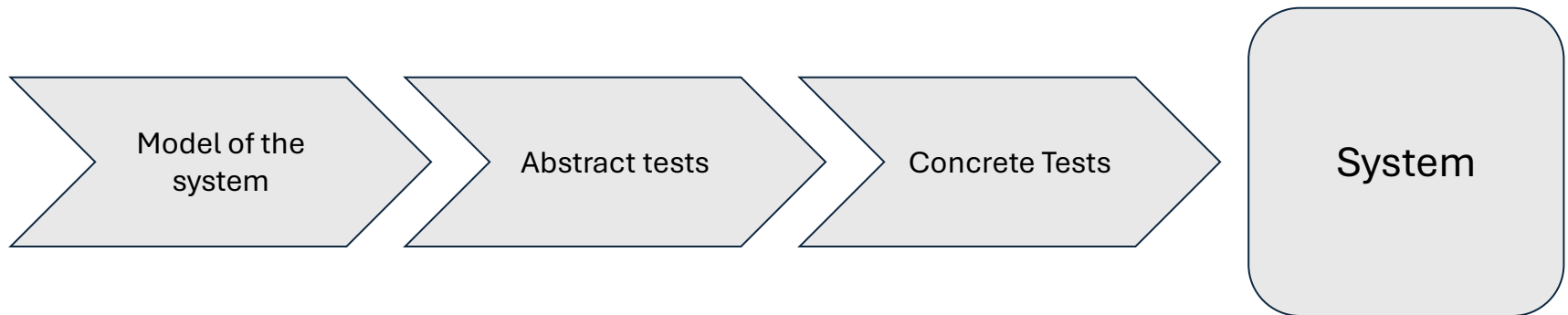
```
  Post     => X = X'Old + 1; -- check if X is incremented
```

```
  -- from its value before the call
```

[<https://learn.adacore.com/courses/intro-to-spark/chapters/01_Overview.html>](https://learn.adacore.com/courses/intro-to-spark/chapters/01_Overview.html)

Model-based testing

- Using models of expected behaviour to produce test-cases that can be used to find discrepancies between actual program behaviour and the model



Random (fuzz) testing

- Testing by feeding in random inputs until program crashes
- Goal: could this result possible to be right?
- Advantage: it ignores the knowledge about how the program works

Boundary testing

- Testing by choosing inputs that are on semantically-significant (extreme) boundaries (ends)
- Boundary values for
 - Boolean
 - Character
 - Numeric (integer, float, double, long, unsigned, etc)
 - String

Coverage-based testing

- Testing until some measure of coverage meets a pre-agreed target
- You want at least one test that involves every:
 - Package/module
 - Class
 - Method
 - Statement
 - Exit (true/false) from every decision logic (if, while, case, etc)
 - Path through the program
- For critical software, you need 100% coverage at unit testing ([1], page 36)

[1] <https://esamultimedia.esa.int/docs/industry/SME/2005-Training/2-SWE-course/ECSS-Q-80B-10October2003.pdf>

Statement coverage

- Definition: a test suite archives statement coverage if it results in each statement in the program being executed at least once

- Example

```
public int returnIncrementEven(int n){
    int result = n;
    if(result%2==0) (++result);
    return result;
}
@Test
public void statementCoveragereturnIncrementEven() {
    assertEquals(5, returnIncrementEven(4)); // 100 percent coverage
    assertEquals(3, returnIncrementEven(3)); // 75 percent coverage
}
```

Branch coverage

- Definition: a test suite archives statement coverage if it results in each branch in the program being taken at least once
- There is another coverage criteria based on this: condition coverage

- Example

```
public int returnIncrementEven(int n){
    int result = n;
    if(result%2==0) (++result);
    return result;
}

@Test
public void branchCoveragereturnIncrementEven(){
    assertEquals(5, returnIncrementEven(4)); // branch 1
    assertEquals(3, returnIncrementEven(3)); // branch 2
}
```

Path coverage

- Definition: a test suite (collection of test cases) archives path coverage if it results in each path through the program being taken at least once
- Very expensive: 10 statements would be 1024 paths
- Example

```
public int returnIncrementEven(int n){  
    int result = n;  
    if(result%2==0) (++result);  
    return result;  
}  
@Test  
public void pathCoverageReturnIncrementEven(){  
    assertEquals(5, returnIncrementEven(4));  
    assertEquals(3, returnIncrementEven(3));  
}
```

Statement vs branch coverage vs path coverage

- Not the same
- Path coverage ensures that each control flow path has been taken once at least
- Branch coverage ensure that each individual branch (alternative) has been taken once at least
 - There may be combinations of branches that haven't been taken
- Summary
 - Path coverage (stronger than) branch coverage
 - Branch coverage (stronger than) statement coverage

Fault injection testing

- Testing by deliberately introducing faults into the software
- After injection (usually a single injection per version), tests are executed against the changed version (called mutant version)
- Tests are then checked if it detects the change

Fault injection testing (cont.d)

- Example

```
public boolean isPalindrome(String word) {  
    if (word.length() == 1) return true;  
    boolean check = word.charAt(0) == word.charAt(word.length() - 1);  
    return check && isPalindrome(word.substring(1, word.length() - 1));  
}  
  
@Test  
public void Test1() {  
    assertFalse(isPalindrome("No")); // pass  
}  
  
@Test  
public void Test2() {  
    assertTrue(isPalindrome("NoN")); // pass  
}
```

Fault injection testing (cont.d)

- Injecting errors (mutations)

```
public boolean isPalindrome(String word) {                                // mutation 1
    if (word.length() >= 1) return true;
    ...
}

public boolean isPalindrome(String word) {                                // mutation 2
    if (word.length() <= 1) return true;
    ...
}

public boolean isPalindrome(String word) {                                // mutation 3
    if (word.length() != 1) return true;
    ...
}

public boolean isPalindrome(String word) {                                // mutation 4
    if (word.length() > 1) return true;
    ...
}
```